Trailmark

Round 1: The Plan

WCC Forge 48 (We Code Coders)
Team: Trailmark (Kishan S, Tharun Sridhar), VIT Vellore

1. Your understanding of the problem

Every project depends on people, and people leave. A student graduates, a volunteer gets busy, an employee resigns. When they go, the files stay behind, but a lot of what they knew does not. They know why a decision was made, what they were halfway through, which part quietly depends on which other part, what usually breaks and how they fix it, and who to ask when something goes wrong. Almost none of that is written down properly.

The information does exist in pieces. A reason is buried in a chat thread, a half-finished task sits on a board, a hint is in a commit message, an agreement was made in a meeting nobody took notes for. So the knowledge is not gone, it is scattered, and the person who knew where everything was is the one leaving.

Who is affected

- The departing member. They are asked to hand over everything in their last days, usually without a clear idea of what the project actually needs from them, so important things get left out or rushed.
- The incoming member. They inherit the work without the context behind it and have to rebuild it from fragments.
- The project lead. They are responsible for continuity, but cannot see which parts of the project are well documented and which depend only on the person who is leaving.

Consequences when knowledge is lost

- The incoming member loses time working out what was already decided, repeats mistakes that were already solved, and may undo or redo work because the reason behind it is unknown.
- Unfinished work is picked up in the wrong state, or dropped because nobody realises it was in progress.
- The remaining team gets pulled into constant questions, guesses about the past and rework, and the project slows down. If the leaver is hard to reach, some questions can never be answered at all.
- Hidden dependencies and recurring problems surface late, usually as a failure at a bad moment.

Why existing documentation does not fix this

Documentation is usually written while the work is going on, for people who already understand the project. It says what was done but rarely why. It goes out of date without anyone noticing. A handover note written in the last week is rushed and only covers what the leaver remembers at that moment, and nobody checks it for gaps.

Why a general AI summary carries its own risk

A summary produced from a large amount of material can read smoothly while hiding where each statement came from. If the sources are incomplete, such a tool may produce an answer that sounds right but is not supported, and the reader has no easy way to tell. For a handover this is a real risk, because the successor is likely to trust the text and act on it.

The core problem as we see it

The project cannot tell what it actually knows. Nobody can say which parts of a handover are supported by evidence, which are only one person's recollection, and which are missing or contradict each other. Until those three are told apart, someone leaving is a risk the team cannot measure. A useful handover has to separate:

- **Verified facts:** statements backed by a source such as a commit, a document, a message or a task record.
- **Unverified statements:** things the departing member says, which are useful but rest only on their word until confirmed.
- **Missing or contradictory information:** gaps where nothing was found, or where sources disagree.

The part we have chosen to solve

Trailmark goes through the project's authorised sources and works out the current state of the project. It finds the places where knowledge is missing, unclear or contradictory, and asks the departing member about those specific points, showing them the evidence that raised each question. Their answers are kept and labelled as unverified statements, separate from what the sources prove. A responsible person, such as the project lead, then reviews, corrects and approves the result, and only then does the incoming member receive a handover that links each claim to its source and shows only what they are permitted to see.

Intended outcome

The aim is faster project continuation. The incoming member should be able to start with fewer unresolved questions, and everything they are told should be backed by evidence or clearly marked as unverified or unknown. The remaining team should spend less time answering questions about the past and more time on the work.

Our focus setting is a small software project run by students or volunteers, where people change often and documentation is thin. The same approach applies to workplaces and research groups.

2. The users affected

Primary user: the incoming member of a student or volunteer software project

This is who we build for. They are a junior student, a new volunteer or a teammate taking over, and they start without the history. They need the current state, the reasons behind decisions, the risks and the next actions, with each statement linked to its source and uncertainty marked. The handover succeeds or fails on whether this person can continue the work.

Supporting users

- **Departing member (a graduating student, a volunteer stepping back):** has little time and does not know what the project will miss. They answer specific questions, each shown with the evidence that prompted it, instead of being told to "write everything down".
- **Project lead or reviewer (club lead, organiser):** is responsible for continuity but cannot see which parts depend on one person. They review, correct, reject and approve the handover before it is shared, and decide who may see what.

Example: resuming an unfinished task

A graduating student leaves a half-built login feature. The incoming member opens the handover and sees the task, its last known state (token refresh not finished, linked to the last commit and issue), the reason the team chose that approach (linked to a chat thread), and one open question marked "unverified: stated by the leaver, no source found". They know where to resume, why the

design is this way, and what still needs confirming, without messaging the leaver first.

Access restrictions

The system applies access rules to what the incoming member is shown, not to what it can read. Connected sources may hold broader information, such as private chats, credentials or other people's personal details. The handover shows only the parts the incoming member is authorised to see, and the rest is left out or redacted.

Scenarios

- **Departure:** a tech lead graduates; an organiser leaves mid-campaign; a developer resigns with two weeks left.
- **Onboarding:** a full replacement joins; someone takes over only part of the work; the successor has a follow-up question after starting.

Environment

Small to mid-sized teams whose activity is spread across a repository, chat, a task board and shared documents, where people change often, documentation is thin and some information is restricted. For the event we use a student or volunteer software project with synthetic demonstration data, so no confidential material is involved.

3. Your proposed solution

Trailmark is a handover platform that finds what a project is missing, asks the departing member about it, and gives the incoming member a reviewed, evidence-linked plan to continue from.

How it works, in short

1. **The project lead connects the sources the team has authorised:** the code repository, documents, chat exports and the task board.
2. **The system reads them and rebuilds the project's current state:** decisions and their reasons, who owns what, unfinished work, dependencies, risks and recurring problems.
3. Every statement in that picture is tagged with where it came from and how well it is supported:
   - **Verified:** backed by a source, shown with the link.
   - **Unverified:** stated by the departing member, with no source to confirm it.
   - **Missing or conflicting:** nothing was found, or sources disagree. The system says so and does not fill the gap.
4. For each gap, the system asks the departing member a specific question and shows the evidence that raised it. For example: "This commit rewrote the payment retry logic, but no reason is recorded. Why was it changed?" Their answers are saved as unverified statements.
   If the departing member is unavailable or never answers, those gaps are not filled in. They stay in the handover as open gaps marked "no answer from the departing member", and the project lead can assign each one to another person who may know, or accept it as a known unknown.
5. The project lead reviews the draft, corrects it, rejects wrong claims, resolves conflicts and approves it. Versions are kept.
6. The incoming member receives the approved handover, filtered to what they are authorised to see, with a clear list of what they own, what is risky, what is still open and what to do first. They confirm receipt. In the final round they can also raise follow-up questions on any statement.

What makes it useful

- The departing member answers a handful of targeted questions instead of writing a long document from memory.

- The incoming member can check any statement against its source, and knows which parts are solid and which are not.
- The project lead sees where the project depends on one person before that person is gone.
- Nothing reaches the successor until a person has approved it.

What makes it different from a general summarising tool

- It is organised around gaps and trust, not around a summary. The question "what do we not know?" is as important as "what do we know?".
- The three-way labelling (verified, unverified, missing or conflicting) appears everywhere in the handover.
- The questions to the leaver come from the evidence, so they are specific to this project.
- Access rules apply to what each person is shown, not just to what the system can read. They cover generated summaries and derived information as well as the original documents: a summary or extracted statement inherits the restrictions of every source it draws on, and is hidden or redacted for anyone not permitted to see all of them.

Scope for the event

We will build this end to end for one setting, a student or volunteer software project, using synthetic data from a repository, documents, a chat export and a task list. Live integrations with every tool and fully automatic approval are out of scope: a person always approves.

4. The complete product workflow

The workflow runs from connecting sources to an approved handover that the incoming member acknowledges. Roles: project lead (also the reviewer), departing member, incoming member.

1. Connect (project lead)

The lead creates a handover for the departing member and the incoming member, then connects the authorised sources: a code repository, documents, a chat export and a task list. For each source the lead sets who may see it. Nothing outside these sources is read. Every piece taken from a source keeps that source's access level.

2. Reconstruct (system)

The system splits each source into small pieces and keeps a link to where each piece came from. It then extracts the project's state: decisions and their reasons, owners, unfinished work, dependencies, blockers, risks, recurring problems and key contacts.

3. Verify (system)

Each extracted statement is checked against its source and given one of three labels:

- **Verified:** supported by a source, shown with the link.
- **Unverified:** stated by the departing member only.
- **Missing or conflicting:** nothing was found, or two sources disagree.

A statement without a valid source link cannot be labelled verified. A summary or extracted statement inherits the restrictions of every source it draws on, and the system records which sources those are. Before anything is shown, it is checked against the viewer's permissions: if the viewer is not allowed to see all the sources behind a statement, the statement is hidden or redacted, even though the text itself was generated and is not a copy of the restricted source. Restricted content is also kept out of any text prepared for a viewer who cannot see it. The system lists the gaps, ordered by risk, such as an unfinished task with no owner.

4. Question the departing member (system, departing member)

For each gap the system asks a specific question and shows the evidence that raised it. The departing member answers, skips, or says they do not know. Answers are stored as unverified statements. Skipped questions stay on the list as open gaps in the handover.

5. Review and approve (project lead)

The lead sees the draft with its labels and sources. They can edit a statement, reject it, resolve a conflict by choosing one source, or mark a gap as accepted. Choosing one source requires a written reason. The other source is not deleted: it stays in the version history, marked as not chosen, with the reason beside it. Every change is saved as a new version with the editor's name. Only an approved version can be shared.

6. Hand over (incoming member)

The incoming member receives the approved handover, filtered to what their role allows. Restricted or sensitive parts are removed or redacted. The handover shows what they now own, the current state of unfinished work, risks and dependencies, key contacts, open questions and a suggested order of next actions. This order is a recommendation for the lead and the incoming member to review and change, not an automatic decision. Each statement links to its source.

7. Acknowledge and follow up (incoming member)

The incoming member confirms they have received the handover. In the final round, if something is unclear, they can raise a question that stays attached to that statement. The lead or the departing member can answer, and the answer is added as a new version.

What each person sees

- **Departing member:** the questions and the evidence behind them.
- **Project lead:** the full draft, all labels, all gaps and the version history.
- **Incoming member:** only the approved handover, limited by their permissions.

Failure cases the workflow handles

- **A source is empty or unreadable:** the system reports it and does not guess.
- **The departing member cannot answer:** the gap stays open and visible.
- **Sources conflict:** both are shown until the lead resolves it.
- **The lead does not approve:** nothing is shared.

5. The main features you plan to build

Core features (working by Round 2)

1. **Source import:** a code repository (commits, pull requests, issues), documents, a chat export and a task list, each with a source access level set by the project lead.
2. **Project-state reconstruction:** decisions and their reasons, owners, unfinished work, dependencies, risks and key contacts, each kept with a link to its source.
3. **Evidence labels:** every statement is marked verified, unverified or missing/conflicting. A statement with no valid source link cannot be marked verified.
4. **Gap detection and questions:** gaps are listed by risk, and each becomes a specific question for the departing member that shows the evidence behind it. Unanswered questions stay visible as open gaps.
5. **Review and approval:** the project lead can edit, reject, resolve conflicts (with a written reason) and approve. Every change is saved as a version.
6. **Permission-aware handover:** the incoming member sees only the approved handover, filtered

to their permissions. Summaries and derived statements inherit the restrictions of their sources, and restricted parts are hidden or redacted.

7. **Handover view with receipt:** what they own, unfinished work, risks, dependencies, open questions and a recommended order of next actions, each with source links, and a button to confirm receipt.

Planned improvements for Round 3

1. **Follow-up questions:** the successor can ask a question on any statement; answers from the lead or departing member are added as new versions.
2. **Decision timeline:** a view of key decisions, changes and unresolved discussions over time, with conflicts shown side by side.
3. **Audit log and stronger access controls:** a record of who viewed, edited and approved what, plus tests that restricted information does not appear in summaries.

Deliberately not included

- Live integrations with every tool, such as Slack or Jira; we use exports and the repository's public interface.
- **Automatic approval:** a person always approves.
- **Anything that fills a gap with a guess:** missing information is shown as missing.

6. Your technology stack and why you chose it

Stack

- **Frontend:** Next.js (React). Quick to build the review, question and handover screens, and easy to deploy.
- **Backend:** Python with FastAPI. Python has the best tools for text and AI work, and FastAPI gives a clean, fast API with little code.
- **Database:** PostgreSQL with the pgvector extension. One database holds claims, source links, versions, users and permissions, and supports similarity search over source text, so permissions can be enforced in the same queries.
- **AI model:** a large language model through an API (for example Claude), used for extraction, gap detection and question writing, with structured output.
- **Source parsing:** the GitHub REST API for repository data, and parsers for markdown, PDF, text, CSV and chat-export files.
- **Hosting:** Vercel for the frontend, Render for the backend and database.
- **Containers:** a minimal Docker setup (backend, frontend and database, run together with Docker Compose) so the project can be started with one command on any machine. The product also runs without it, so it is a convenience and not a dependency.

Authentication and access control

- **Sign-in:** email and password. Passwords are stored only as salted hashes (argon2). A successful sign-in returns a short-lived signed token (JWT) kept in an HTTP-only, secure cookie, so scripts in the page cannot read it.
- **Every API request goes through one backend check:** the token is verified, the user is loaded, and the user's role and membership for that specific handover are looked up in the database. A request with no valid token is rejected with 401, and a request outside the user's role or handover is rejected with 403.
- Access is enforced on the server, never in the frontend. Source pieces and claims carry an access level, and the database queries that return them filter by the caller's permissions. PostgreSQL row-level security is turned on as a second layer, so a coding mistake in one endpoint does not

expose restricted rows.
- Derived statements store the sources they came from and are hidden or redacted when the viewer cannot see all of them.

How claim support is checked

Confirming that a source ID exists does not show that the source supports the claim, so we check in three steps:

1. The model must return, with each claim, the source IDs and the exact quoted text it relied on. The backend confirms the quote appears word for word in that source piece. A fabricated quote fails here.
2. A separate verification call, with its own prompt, sees only the claim and the quoted text and answers: supported, partly supported or not supported. Only "supported" can be labelled verified. The others become unverified or missing.
3. The project lead reviews every claim before approval and can see the quote beside it.

This reduces unsupported claims but cannot prove correctness, which is why the human review step stays and why we state this limit openly. We will test it on our synthetic project, where we know which claims are true.

If the AI service fails or runs out of free-tier quota

- Requests are retried with a delay, and the system can switch to a second configured model.
- For the demonstration project, extraction results are generated once and saved, so a judge can always open and review the handover without a live call.
- Without any AI, the system still lists the sources, the unanswered tasks, owners from the task list and commit authors, and the open questions. The interface shows a clear notice such as "AI unavailable: showing saved results" and never presents a blank or invented answer.

Secrets and credentials

- API keys and database credentials live only in environment variables held in the Vercel and Render secret stores. They are never written into the code or the repository.
- A .env file is listed in .gitignore, and the repository holds only an .env.example with placeholder values. GitHub secret scanning is switched on to catch any leak.
- The LLM and database are called only from the backend, so keys are never sent to the browser. Secrets are not written to logs.
- The database user for the application has only the permissions it needs. Connections use TLS, and any key that is exposed is rotated immediately.

Hardest technical part

Producing statements that are supported by their sources and never reveal restricted information. The claim checks and the permission design above are aimed at this.

Why this fits the scope

All of these tools are mature and low-cost or free, and our team can build with them within the event. We depend on no private data or hard-to-get access.

Data and external services

- Demonstration data is a synthetic project we create, with a repository, documents, a chat export and a task list that include deliberate gaps and contradictions. It will be disclosed as AI-generated synthetic data.
- **External services:** the LLM API, GitHub, Vercel and Render. AI assistance used in development will be disclosed in the repository.

7. Each team member's responsibilities

Team size: 2

Both members work together across the project. The roles below show who takes the main responsibility for each area, and each member is able to explain the whole system.

Member 1 (team leader)

Full name: Kishan S
Role / area: Team Leader and Frontend Developer
Main responsibilities:

- Lead the overall development and coordinate the project workflow between both team members.
- **Develop the frontend:** the review and approval screen with version history, the incoming member's handover view, the document upload screen and the departing member's questions screen.
- Create the synthetic demonstration project with planted gaps and contradictions, and its answer key.
- Carry out code review, integration testing and debugging, and run permission and leakage tests.
- Handle application deployment, the presentation and final demonstration, and the final project submission.
- **Build the Round 3 improvements:** the decision timeline view, follow-up questions from the incoming member, and the audit log.

Skills / tools: React.js, JavaScript, TypeScript, Python, FastAPI, SQL and PostgreSQL, REST APIs, Git, GitHub and deployment tools. Has built AI agent and chatbot projects in Python and JavaScript.

Member 2

Full name: Tharun Sridhar
Role / area: Backend and AI/ML Engineer
Main responsibilities:

- **Build the AI pipeline:** project-state extraction, evidence labels, claim-support checks, gap detection and questions for the departing member, and a fallback for when the AI service is unavailable.
- Build the backend APIs and database structure to manage projects, users, uploaded documents and handover records, and the importers for the project sources.
- Implement authentication, access control, secure data handling, role-based sharing and the redaction of derived content.
- Set up a minimal Docker configuration (backend, frontend and database) so the project can be run with one command.
- Prepare the documentation and README, and manage the GitHub repository.

Skills / tools: Python, FastAPI, Django REST Framework, Node.js, TypeScript, JavaScript, SQL, PostgreSQL, Redis, JWT authentication and role-based access control, REST API design, LLM integration (Gemini, Groq, OpenAI), prompt engineering, RAG pipelines, LangGraph, LangChain, MCP, Docker, GitHub Actions, AWS, Git, pytest.

Work the whole team shares

- Both members review each other's work so that critical tasks are cross-checked.
- Testing the system with synthetic project data, validating AI-generated results and improving the overall user experience.

Who is responsible for each of these

- **Frontend and screens:** Kishan S
- **Synthetic demonstration data:** Kishan S
- **Code review, integration testing and debugging:** Kishan S
- **Permission and leakage tests:** Kishan S
- **Deployment:** Kishan S
- **Presentation and final demonstration:** Kishan S
- **Final submission:** Kishan S
- **Round 3 improvements (decision timeline, follow-up questions, audit log):** Kishan S
- **AI pipeline (extraction, evidence labels, questions, fallback):** Tharun Sridhar
- **Backend, APIs, database and importers:** Tharun Sridhar
- **Authentication and access control:** Tharun Sridhar
- **Docker setup:** Tharun Sridhar
- **Documentation and README:** Tharun Sridhar
- **GitHub repository management:** Tharun Sridhar

If a person leaves or is unavailable, who covers their work

Both team members will maintain shared documentation and understand the overall system architecture. Tharun Sridhar will keep the frontend running, and Kishan S will learn the backend APIs and application workflow so he can maintain them. Critical tasks will be cross-reviewed so that development can continue without depending entirely on one person.

8. Your development plan across all three rounds

Round dates for Rounds 2 and 3 have not been announced, so the plan is given in stages. We will fit it to the official build window and will not start building before it opens.

Round 1: The Plan (current)

- Define the problem, users, workflow, features, stack and team roles (this document).
- **Decide the demonstration setting:** a synthetic student or volunteer software project.

Round 2: The Build (goal: the core workflow works end to end)

Stage 1, foundation

- Set up the repository, backend, database schema, authentication and role checks.
- Create the synthetic project (repository, documents, chat export, task list) with planted gaps and contradictions, and an answer key to test against.

Stage 2, core pipeline

- Build the source importers, with each piece keeping its source link and access level.
- Build state reconstruction, evidence labels, claim-support checks and gap detection with question generation.
- Save the demonstration results so the demo works even if the AI service is unavailable.

Stage 3, human flow

- Build the screens for the departing member's questions, the lead's review and approval with version history, and the incoming member's permission-filtered handover with receipt.

Stage 4, check and submit

- Run the full journey on the synthetic project and test failure cases (empty source, unanswered questions, conflicting sources, restricted content).

- Add a minimal Docker setup so the project can be run with one command.
- Deploy, write the architecture and decisions document, list the APIs, models and external services used, and record test evidence.
- Commit to the three Round 3 improvements.

Round 3: The Finish (goal: a reliable, complete product)

Three committed improvements

1. **Follow-up questions:** the successor can question any statement; answers become new versions.
2. **Decision timeline:** key decisions, changes and unresolved discussions over time, with conflicts side by side.
3. **Audit log and stronger access control:** who viewed, edited and approved what, plus tests that restricted information never appears in summaries.

Also in Round 3

- Fix reliability issues found in Round 2 and improve how missing information and failures are shown.
- Improve usability so a new user can complete a handover with little help, and check accessibility.
- Write the README with setup steps, testing notes, a privacy, security and responsible-design explanation, known limitations and a roadmap.
- Record the demonstration video and prepare the final report.

Risks and how we handle them

- **AI output is unreliable:** the claim checks, human review and saved demonstration results reduce the impact.
- **Free-tier limits on the AI service:** retries, a second model and saved results.
- **Time is short:** the core workflow comes first, and Round 3 items are built only after it is stable.
- **Someone becomes unavailable:** roles have named backups, and work is reviewed by a second person.

9. The expected impact of your solution

The aim is faster project continuation: the incoming member starts work with fewer unresolved questions and information they can check.

Expected impact by user

- **Incoming member:** understands what they own, the state of unfinished work, the risks and the next actions without waiting for the departing member to be available. Each statement can be checked against its source, and uncertain or missing points are marked, so they know where to be careful.
- **Departing member:** answers a short list of specific questions instead of writing a long document from memory, so less of their last days is spent on handover and less is forgotten.
- **Project lead:** sees where the project depends on one person, and approves a handover they have reviewed, instead of trusting an unchecked note.
- **Remaining team:** fewer interruptions about past decisions and less rework from picking up unfinished work in the wrong state.

Wider impact

- Handovers become consistent across people and projects, so the quality no longer depends on how much time the leaver has or how careful they are.

- Gaps become visible before the person leaves, while they can still be asked.
- Sensitive information stays within the people permitted to see it, even when sources are broadly connected.

Targets and how we will measure them

These are goals to test on our synthetic project, not results we already have.

- **Questions the incoming member must ask the departing member after receiving the handover:** fewer than with a plain document or summary.
- **Share of handover statements that carry a valid source link:** every statement marked verified.
- **Known gaps in the synthetic project that the system finds:** measured against our answer key.
- **Restricted information appearing for a person not permitted to see it:** none in our tests.
- **Time for the departing member to complete their part:** measured in testing, with the aim of keeping it short.

Limits

The impact depends on the sources provided, and knowledge that is in no source is captured only if the departing member answers. We will report results from the synthetic project as such and will not claim real-world outcomes we have not measured.

10. Future scope after the event

Planned next steps, in the order we would take them. These are intentions and not features we claim to have built.

1. Live integrations

Connect directly to the tools teams already use, such as Slack, Jira, Notion and Google Drive, with each tool's own permissions carried into the system, instead of relying on exports.

2. Continuous knowledge health

Check a project's knowledge regularly, not only when someone leaves, and warn the lead early about risks such as an area that only one person has ever touched or a decision with no recorded reason.

3. Better handling of conflicts and change

Track how decisions and ownership change over time, flag when a verified statement becomes outdated, and keep handovers current after they are approved.

4. Stronger privacy and deployment options

Add finer-grained access rules, automatic detection of sensitive data, and a self-hosted option for organisations that cannot send data to an external AI service.

5. Wider settings and languages

Adapt the workflow for workplaces, research groups and volunteer communities with different source types, and support more than one language.

6. Measured improvement

Run pilots with real teams, with their permission and appropriate data handling, to measure how much the handover reduces questions and onboarding time, and use the findings to improve the

system.

11. Complete team and participant details

Team name (exactly as entered in the submission form): Trailmark
Project name: Trailmark
Event: WCC Forge 48 (We Code Coders)
Team size: 2

Team leader (submits the form)

Full name: Kishan S
Email: kishansadeesh13@gmail.com
College / organisation: Vellore Institute of Technology (VIT), Vellore
Course / year or role: 2026 batch
GitHub username: KishanSadeesh (https://github.com/KishanSadeesh)

Member 2

Full name: Tharun Sridhar
Email: tharunsridhar@gmail.com
College / organisation: Vellore Institute of Technology (VIT), Vellore
Course / year or role: B.Tech Computer Science (Information Security), 2026 batch
GitHub username: tharunsridhar (https://github.com/tharunsridhar)

GitHub repository (single repository for this project)

Repository link: https://github.com/try2mastercode/Trailmark
Visibility: Public
Repository owner account: try2mastercode (https://github.com/try2mastercode). This account belongs to Tharun Sridhar and is used only to own the repository.
Commit account of Tharun Sridhar: tharunsridhar (https://github.com/tharunsridhar). This is Tharun Sridhar's personal GitHub account. His commits to the repository appear under this username.
Commit account of Kishan S: KishanSadeesh (https://github.com/KishanSadeesh). He is added as a contributor to the repository, and his commits appear under this username.
In short: the repository is owned by try2mastercode, and all commits come from the two team members' own accounts, tharunsridhar (Tharun Sridhar) and KishanSadeesh (Kishan S). There are two team members and no other contributors.

Google Drive folder: https://drive.google.com/drive/folders/15xtvkU-fjcVt4eayFnpqd4ZE9N5iZMFs?usp=sharing

Declarations

- Each member belongs to only this team in this event.
- The details above are accurate.
- Demonstration data is synthetic, and AI assistance used in development will be disclosed in the repository.